

华东师范大学软件工程学院实验报告

姓名：高珩博

学号：10255101445

实验编号：Lab4

实验名称：Cathé 实验

一、实验目的

实验需要在 linux 环境下修改两个源文件代码，实现相应的功能。实验分为两部分，第一部分要理解 Cache 的原理，编写代码模拟 Cache 的行为；第二部分需要优化一个矩阵转置函数，最大程度地减少高速缓存未命中的次数。

二、实验内容与实验步骤

1. Cache simulator:

这部分需要我们模拟高速缓存，该模拟器以 valgrind 内存跟踪为输入，在该跟踪上模拟缓存的命中/未命中行为，并输出命中，未命中和逐出的总数。

运行前先运行 make clean 和 make 指令生成可执行文件。参考模拟器采用以下命令行参数：

Usage: ./csim-ref [-hv] -s <s> -E <E> -b -t <tracefile>

-h: 可选的帮助标志，用于打印使用情况信息

-v: 显示跟踪信息的可选详细标志

-s <s>: 设置的索引位数 ($S = 2^s$ 是设置的数量)

-E <E>: 关联性 (每组行数)

-b : 块位数 ($B = 2^b$ 是块大小)

-t <tracefile>: 要重播的 valgrind 跟踪的名称

PPT 中给出了 getopt 和 fscanf 的用法：getopt 用于解析命令行参数，fscanf 可以从文件中读取数据；使用 malloc/free 进行空间管理。直接使用以上指令即可。

根据讲义，缓存行的结构体只需要包括有效位，标记位，时间戳三个变量就够了，之后我们考虑初始化一个[S][E]的二位数组，完成对 cache 的模拟。

接着分析每个输入的指令应该被如何操作。如果是 I，则不是数据操作，直接忽略。如果是 L 或者 S，则需要进行一次 hit-miss eviction 检测，如果是 M，则相当于先 L 再 S，需要进行两次 hit-miss- eviction 检测。然后考虑 hit-miss- eviction 检测细节。即代码如下：

```
while(fscanf(file, " %c %x,%d", &operation, &address, &size) > 0){
    switch(operation){
        case 'L':
            update(address);
            break;
        case 'M':
            update(address);
        case 'S':
            update(address);
            break;
    }
    time();
}
```

读取的结构地址如下：低 b 位表示 block 偏移，本实验中不需要计算 blk 偏移。中间 s 位是 set index 位，表示对那个行操作。其余 t 位是 tag 位。用于标明对应的 line 是否有效。我们需要对得到的地址进行如下操作，解析出 t 和 s。

unsigned s_address = (address >> b) & ((0xffffffff) >> (32 - s));

unsigned t_address = address >> (s + b);

替换策略使用的是 LRU 的缓存替换策略。如果该 SET 存满了，每次要找时间戳最小的替换。为了方便，时间戳初始化为 0，之后每个操作+1。当 TIMESTAMP = 0 的时候就代表无效。

而 cache 的容量有限，当满的时候需要驱逐某行。先遍历当前组，判断它满了没有，可以遍历所有的行，只要有一个有效位为 0，（有效位的作用是说明该行是否存储了数据，通俗的理解就是是否为空）则该组未滿。

综上所述，完整的实现代码 请见附录

运行评测程序

```

boyu@LAPTOP-0A8VIRCU:~/lab4/cachelab-handout$ ./test-csim
Your simulator      Reference simulator
Points (s,E,b) Hits Misses Evicts Hits Misses Evicts
3 (1,1,1) 9 8 6 9 8 6 traces/yi2.trace
3 (4,2,4) 4 5 2 4 5 2 traces/yi.trace
3 (2,1,4) 2 3 1 2 3 1 traces/dave.trace
3 (2,1,3) 167 71 67 167 71 67 traces/trans.trace
3 (2,2,3) 201 37 29 201 37 29 traces/trans.trace
3 (2,4,3) 212 26 10 212 26 10 traces/trans.trace
3 (5,1,5) 231 7 0 231 7 0 traces/trans.trace
6 (5,1,5) 265189 21775 21743 265189 21775 21743 traces/long.trace
27
TEST_CSIM_RESULTS=27

```

成功

2. Efficient Matrix Transpose:

需要在 `trans.c` 中编写转置函数，该函数将尽可能提高高速缓存命中率。在源文件中提供了一个示例转置函数，该函数计算 $N \times M$ 矩阵 A 的转置并将结果储存在 $M \times N$ 矩阵 B 中。

示例函数的未命中率很高，我们将在 `transpose_submit` 中编写函数，尽可能降低未命中率。缓存采用的是直接映射高速缓存， $s = 5$ ， $b = 5$ ， $E = 1$ 。对于该缓存，总共存在 32 个组，每个组共 32 个字节，可以装入 8 个 `int` 型变量，是非常有限的缓存， A 和 B 会映射到相同的块，使用传统转置方法必然会有很高的未命中率，所以决定对矩阵进行分块，同时为了减少对矩阵的访问次数以从根源上减少为命中次数，使用中间变量暂存。

(1) 32*32

一个 `int` 类型数字是 4 字节，cache 中一行 32 字节，可以放 8 个 `int`。通过使用分块技术，我们 cache 一行能放 8 个，所以我们分块最好也能用 8 的倍数。然后对于非对角线上的块，本身就没有额外的冲突；对于对角线上的块，写入 A 每一行的第一个元素后，这一行的元素都进入了缓存，我们就立即用本地变量存下这 8 个元素，随后再复制给 B 。这样，就避免了第一个元素复制时， B 把 A 的缓冲行驱逐，导致没有利用上 A 的缓冲。

将矩阵分成 4 个 8×8 子矩阵，在循环中按照行列进行枚举，每次对变量进行赋值的时候，对于 A 的赋值，每八次只会发生一次 miss，对于 B 的赋值，第一次八个全不命中，后续大部分都能够命中。

代码见附录

运行评测程序：

```

boyu@LAPTOP-0A8VIRCU:~/lab4/cachelab-handout$ ./test-trans -M 32 -N 32
Function 0 (2 total)
Step 1: Validating and generating memory traces
Step 2: Evaluating performance (s=5, E=1, b=5)
func 0 (Transpose submission): hits:1765, misses:288, evictions:256

Function 1 (2 total)
Step 1: Validating and generating memory traces
Step 2: Evaluating performance (s=5, E=1, b=5)
func 1 (Simple row-wise scan transpose): hits:869, misses:1184, evictions:1152

Summary for official submission (func 0): correctness=1 misses=288
TEST_TRANS_RESULTS=1:288

```

小于 300 成功

(2) 64*64

数组一行有 64 个 `int`，即 8 个 block，所以每四行就会填满一个 cache，即两个元素相差四行就会发生冲突，所以我们设置块大小为 4，其余内容模仿 32*32 即可。

代码见附录

运行评测程序：

```

boyu@LAPTOP-0A8VIRCU:~/lab4/cachelab-handout$ ./test-trans -M 64 -N 64
Function 0 (2 total)
Step 1: Validating and generating memory traces
Step 2: Evaluating performance (s=5, E=1, b=5)
func 0 (Transpose submission): hits:9073, misses:1172, evictions:1140

Function 1 (2 total)
Step 1: Validating and generating memory traces
Step 2: Evaluating performance (s=5, E=1, b=5)
func 1 (Simple row-wise scan transpose): hits:3473, misses:4724, evictions:4692

Summary for official submission (func 0): correctness=1 misses=1172
TEST_TRANS_RESULTS=1:1172

```

结果 1172 小于 1300 成功

(3) 61*67

对能够被 8 整除的区域做 8×8 分块处理，对剩余边缘元素做逐个处理，分块区域命中效果与前文一样，边缘区域逐个访问未命中次数可能偏多。

代码见附录

运行评测程序:

```
boyu@LAPTOP-0A8VIRCU:~/lab4/cachelab-handout$ ./test-trans -M 61 -N 67

Function 0 (2 total)
Step 1: Validating and generating memory traces
Step 2: Evaluating performance (s=5, E=1, b=5)
func 0 (Transpose submission): hits:6333, misses:1906, evictions:1874

Function 1 (2 total)
Step 1: Validating and generating memory traces
Step 2: Evaluating performance (s=5, E=1, b=5)
func 1 (Simple row-wise scan transpose): hits:3755, misses:4424, evictions:4392

Summary for official submission (func 0): correctness=1 misses=1906

TEST_TRANS_RESULTS=1:1906
```

小于 2000, 成功

最后, 运行总的评分程序

截图为:

```
boyu@LAPTOP-0A8VIRCU:~/lab4/cachelab-handout$ cd /home/boyu/lab4/cachelab-handout && python3 driver.py
Part A: Testing cache simulator
Running ./test-csim

```

Points (s,E,b)	Your simulator			Reference simulator			
	Hits	Misses	Evicts	Hits	Misses	Evicts	
3 (1,1,1)	9	8	6	9	8	6	traces/yi2.trace
3 (4,2,4)	4	5	2	4	5	2	traces/yi.trace
3 (2,1,4)	2	3	1	2	3	1	traces/dave.trace
3 (2,1,3)	167	71	67	167	71	67	traces/trans.trace
3 (2,2,3)	201	37	29	201	37	29	traces/trans.trace
3 (2,4,3)	212	26	10	212	26	10	traces/trans.trace
3 (5,1,5)	231	7	0	231	7	0	traces/trans.trace
6 (5,1,5)	265189	21775	21743	265189	21775	21743	traces/long.trace

```
27

Part B: Testing transpose function
Running ./test-trans -M 32 -N 32
Running ./test-trans -M 64 -N 64
Running ./test-trans -M 61 -N 67

Cache Lab summary:

```

	Points	Max pts	Misses
Csim correctness	27.0	27	
Trans perf 32x32	8.0	8	288
Trans perf 64x64	8.0	8	1172
Trans perf 61x67	10.0	10	1906
Total points	53.0	53	

三、实验过程与分析

过程已在“实验内容与步骤”中详细描述, 实验过程较为顺利, 唯一出现问题的地方在于最终评测程序, 一开始运行不通, 于是借用 ai 的大手 对评测文件进行了修改, 最终成功出现了结果

四、实验结果总结

总的来说, 实验过程比较顺利, 有不懂的地方也查询资料和询问 ai 解决了

Lab4 确实是比前几个实验简单一点 (上次 lab3 快给我看晕了。。)

通过这个实验, 让我进一步了解了高速缓存运行的机制, 使我对缓存机制有了更清楚的认知。

五、附录 (源代码)

1:

```
#include "cachelab.h"
#include <getopt.h>
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>
#include <stddef.h>
typedef struct {
    int valid_bits;
    unsigned tag;
    int stamp;
} cache_line;
char* filepath = NULL;
```

```

int s,E,b,S;
int hit=0,miss=0,eviction=0;
cache_line** cache = NULL;

void init(){
    cache = (cache_line**)malloc(sizeof(cache_line*)*S);
    for(int i=0;i<S;i++){
        *(cache+i) = (cache_line*)malloc(sizeof(cache_line)*E);
        for(int i=0;i<S;i++){
            for(int j=0;j<E;j++){
                cache[i][j].valid_bits = 0;
                cache[i][j].tag = 0xffffffff;
                cache[i][j].stamp = 0;
            }
        }
    }
}

void update(unsigned address){
    unsigned s_address =(address>>b) & ((0xffffffff)>>(32-s));
    unsigned t_address = address>>(s+b);
    for(int i=0;i<E;i++){
        if(*(cache+s_address+i)->tag ==t_address){
            cache[s_address][i].stamp = 0;
            hit++;
            return;
        }
    }
    for(int i=0;i<E;i++){
        if(cache[s_address][i].valid_bits == 0){
            cache[s_address][i].tag = t_address;
            cache[s_address][i].valid_bits = 1;
            cache[s_address][i].stamp = 0;
            miss++;
            return;
        }
    }
    int max_stamp=0;
    int max_i;
    for(int i=0;i<E;i++){
        if(cache[s_address][i].stamp > max_stamp){
            max_stamp = cache[s_address][i].stamp;
            max_i = i;
        }
    }
    eviction++;
    miss++;
    cache[s_address][max_i].tag = t_address;
    cache[s_address][max_i].stamp = 0;
}

void time(){
    for(int i=0;i<S;i++){
        for(int j=0;j<E;j++){

```

```

        if(cache[i][j].valid_bits == 1)
            cache[i][j].stamp++;
    }
}
}
int main(int argc,char *argv[])
{
    int opt;
    while((opt = getopt(argc,argv,"s:E:b:t:")) !=-1){
        switch(opt){
            case 's':
                s=atoi(optarg);
                break;
            case 'E':
                E=atoi(optarg);
                break;
            case 'b':
                b=atoi(optarg);
                break;
            case 't':
                filepath = optarg;
                break;
        }
    }
    S = 1<<s;
    init();
    FILE* file=fopen(filepath,"r");
    if(file == NULL){ // read trace file
        printf("Open file wrong");
        exit(-1);
    }
    char operation;
    unsigned address;
    int size;
    while(fscanf(file," %c %x,%d",&operation,&address,&size)>0){
        switch(operation){
            case 'L':
                update(address);
                break;
            case 'M':
                update(address);
            case 'S':
                update(address);
                break;
        }
        time();
    }
    for(int i=0;i<S;i++)
        free(*(cache+i));
    free(cache);
    fclose(file);
    printSummary(hit,miss,eviction);
    return 0;
}

```

}

2:

```
void transpose_submit(int M, int N, int A[N][M], int B[M][N])
{
    int i, j, k, l, a0, a1, a2, a3, a4, a5, a6, a7;
    if(M == 32){
        for (i = 0; i < N; i+=8) {
            for (j = 0; j < M; j+=8) {
                if(i == j){
                    for(k = i ;k < i + 8 && k<N;k++){
                        a0 = A[k][j];
                        a1 = A[k][j+1];
                        a2 = A[k][j+2];
                        a3 = A[k][j+3];
                        a4 = A[k][j+4];
                        a5 = A[k][j+5];
                        a6 = A[k][j+6];
                        a7 = A[k][j+7];
                        B[j][k] = a0;
                        B[j+1][k] = a1;
                        B[j+2][k] = a2;
                        B[j+3][k] = a3;
                        B[j+4][k] = a4;
                        B[j+5][k] = a5;
                        B[j+6][k] = a6;
                        B[j+7][k] = a7;
                    }
                }
                else{
                    for(k = i ;k < i + 8 && k<N;k++){
                        for(l = j ; l < j + 8 && l < M;l++)
                            B[l][k] = A[k][l];
                    }
                }
            }
        }
    }
    else if(M == 64){
        for (i = 0; i < N; i += 8) { // 枚举每八行
            for (j = 0; j < M; j += 8) { // 枚举每八列
                for (k = i; k < i + 4; k++) {
                    a0 = A[k][j];
                    a1 = A[k][j + 1];
                    a2 = A[k][j + 2];
                    a3 = A[k][j + 3];
                    a4 = A[k][j + 4];
                    a5 = A[k][j + 5];
                    a6 = A[k][j + 6];
                    a7 = A[k][j + 7];

                    B[j][k] = a0;
```

```

        B[j + 1][k] = a1;
        B[j + 2][k] = a2;
        B[j + 3][k] = a3;

        B[j][k + 4] = a4;
        B[j + 1][k + 4] = a5;
        B[j + 2][k + 4] = a6;
        B[j + 3][k + 4] = a7;
    }
    for (l = j + 4; l < j + 8; l++) {

        a4 = A[i + 4][l - 4];
        a5 = A[i + 5][l - 4];
        a6 = A[i + 6][l - 4];
        a7 = A[i + 7][l - 4];

        a0 = B[l - 4][i + 4];
        a1 = B[l - 4][i + 5];
        a2 = B[l - 4][i + 6];
        a3 = B[l - 4][i + 7];

        B[l - 4][i + 4] = a4;
        B[l - 4][i + 5] = a5;
        B[l - 4][i + 6] = a6;
        B[l - 4][i + 7] = a7;

        B[l][i] = a0;
        B[l][i + 1] = a1;
        B[l][i + 2] = a2;
        B[l][i + 3] = a3;

        B[l][i + 4] = A[i + 4][l];
        B[l][i + 5] = A[i + 5][l];
        B[l][i + 6] = A[i + 6][l];
        B[l][i + 7] = A[i + 7][l];
    }
}
}

/* fallback: block size 16 for other sizes (e.g., 61x67) */
if(M == 61 && N == 67)
{
    int i, j, v1, v2, v3, v4, v5, v6, v7, v8;
    int n = N / 8 * 8;
    int m = M / 8 * 8;
    for (j = 0; j < m; j += 8)
        for (i = 0; i < n; ++i)
        {
            v1 = A[i][j];
            v2 = A[i][j+1];
            v3 = A[i][j+2];

```

```
        v4 = A[i][j+3];
        v5 = A[i][j+4];
        v6 = A[i][j+5];
        v7 = A[i][j+6];
        v8 = A[i][j+7];

        B[j][i] = v1;
        B[j+1][i] = v2;
        B[j+2][i] = v3;
        B[j+3][i] = v4;
        B[j+4][i] = v5;
        B[j+5][i] = v6;
        B[j+6][i] = v7;
        B[j+7][i] = v8;
    }
    for (i = n; i < N; ++i)
        for (j = m; j < M; ++j)
        {
            v1 = A[i][j];
            B[j][i] = v1;
        }
    for (i = 0; i < N; ++i)
        for (j = m; j < M; ++j)
        {
            v1 = A[i][j];
            B[j][i] = v1;
        }
    for (i = n; i < N; ++i)
        for (j = 0; j < M; ++j)
        {
            v1 = A[i][j];
            B[j][i] = v1;
        }
    }
}
```